

Unidade 3

Introdução à Análise de Dados com Python

Aula 1

Aplicação de Banco de Dados com Python

Aplicação de banco de dados com Python

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Python é uma linguagem de programação amplamente utilizada para interagir com sistemas de gerenciamento de banco de dados (SGBD) por meio de bibliotecas como a `sqlite3`. Essa biblioteca permite a criação, leitura, atualização e exclusão de dados em bancos de dados SQL, seguindo o modelo CRUD (Create, Read, Update, Delete). Com a Python, nós, desenvolvedores, podemos desenvolver aplicativos que se comunicam de forma eficaz com bancos de dados relacionais, proporcionando flexibilidade e escalabilidade para nossas aplicações.

O modelo CRUD consiste em uma abordagem fundamental para operações de banco de dados. Nesse contexto, “Create” envolve a inserção de novos registros, “Read” refere-se à recuperação

de informações, “Update” possibilita a modificação de registros existentes e “Delete” diz respeito à exclusão de dados. Python simplifica a implementação dessas operações CRUD, fazendo desse modelo uma escolha popular para desenvolvedores que desejam criar aplicativos com funcionalidades de banco de dados eficientes e robustas.

Suponha, agora, que você precise criar uma tabela de contatos para a comunicação da empresa em que trabalha. Nessa tabela, a seção “Contatos” deve armazenar informações de contatos, incluindo nome, e-mail e número de telefone.

Vamos Começar!

Linguagem de consulta estruturada – SQL

A linguagem SQL (Structured Query Language) desempenha um papel fundamental na comunicação com bancos de dados relacionais. Ela foi inicialmente estabelecida como um padrão pelo American National Standards Institute (ANSI) em 1986 e passou por várias revisões desde então. Embora diferentes fornecedores de softwares de banco de dados, como Oracle e Microsoft, tenham adaptado o SQL com suas extensões e modificações exclusivas, ainda existe um núcleo comum de comandos SQL que é padrão em todos os sistemas.

As instruções em SQL podem ser agrupadas em três categorias principais:

1. DDL (Data Definition Language – Linguagem de Definição de Dados):

- Essas instruções se concentram na estrutura do banco de dados, permitindo a criação, modificação e exclusão de bancos de dados e tabelas. Alguns comandos incluem CREATE (para criar tabelas), ALTER (para modificar a estrutura) e DROP (para excluir tabelas ou bancos de dados).

2. DML (Data Manipulation Language – Linguagem de Manipulação de Dados):

- As instruções DML são usadas para recuperar, atualizar, inserir e excluir dados no banco de dados. Comandos comuns incluem SELECT (para recuperar dados), INSERT (para adicionar novos registros), UPDATE (para modificar registros existentes) e DELETE (para excluir registros).

3. DCL (Data Control Language – Linguagem de Controle de Dados):

- O DCL lida com a segurança e a autorização de acesso aos dados no banco de dados. Comandos como GRANT (para conceder privilégios) e REVOKE (para revogar privilégios) são usados para garantir que apenas usuários autorizados possam acessar e modificar os dados.

Além dessas categorias, o SQL também oferece funcionalidades avançadas, como agregações, junções, subconsultas e transações, as quais viabilizam consultas complexas e a manipulação eficaz de dados. A flexibilidade do SQL o torna uma linguagem poderosa para trabalhar com bancos de dados relacionais, independentemente do sistema de gerenciamento de banco de dados (SGBD) específico em uso. Embora possa haver variações nas implementações de SQL de diferentes fornecedores, a base comum permite que os desenvolvedores escrevam consultas portáteis que funcionam em várias plataformas de banco de dados.

Conexão com banco de dados

Quando desenvolvemos uma aplicação em uma linguagem de programação que precisa interagir com um Sistema Gerenciador de Banco de Dados Relacional (RDBMS), é essencial estabelecer uma conexão entre esses dois processos distintos. Depois que a conexão é estabelecida, podemos enviar comandos SQL para realizar operações no banco de dados. Para viabilizar essa comunicação entre a linguagem de programação e o RDBMS, fazemos uso de tecnologias como Open Database Connectivity (ODBC) e Java Database Connectivity (JDBC).

Tanto o ODBC quanto o JDBC oferecem uma maneira padronizada de os programadores acessarem os recursos do banco de dados a partir de uma Interface de Programação de Aplicativos (API). Uma das grandes vantagens dessas tecnologias é a possibilidade de que uma aplicação acesse diferentes

Sistemas Gerenciadores de Banco de Dados sem a necessidade de recompilar o código. Isso se torna viável porque a comunicação direta com o RDBMS é realizada por meio de um software específico, chamado “driver”, responsável por traduzir as chamadas ODBC e JDBC para a linguagem compreendida pelo RDBMS.

No contexto de Python, para se comunicar com um RDBMS, podemos usar bibliotecas específicas que incorporam os drivers de fornecedores. Isso permite a conexão e a execução de comandos SQL no banco de dados. O PEP 249 (Python Database API Specification v2.0) estabelece regras que os fornecedores devem seguir ao criar módulos para acessar bancos de dados. Um dos princípios é o de que todos os módulos precisam implementar um método chamado “connect(parameters...)” para conceber a conexão com o banco. Isso facilita a alteração do banco de dados, pois apenas os parâmetros de conexão precisam ser ajustados, sem a necessidade de modificar o código.

O SQLite é uma poderosa biblioteca de banco de dados escrita em linguagem C que oferece um mecanismo de banco de dados SQL completo, embora compacto, e de alta confiabilidade. Diferentemente da maioria dos sistemas de gerenciamento de bancos de dados SQL, o SQLite opera sem a necessidade de um servidor separado. Em vez disso, ele lê e escreve diretamente em arquivos de disco. Isso significa que um banco de dados completo, contendo tabelas, índices, triggers e visualizações, é armazenado em um único arquivo no sistema de arquivos. Para os desenvolvedores que utilizam

Python, a linguagem possui um módulo integrado chamado “sqlite3”, o qual permite a interação com o mecanismo do banco de dados SQLite.

Vamos criar um banco de dados! Faremos nossa implementação no Google Colab.

```
import sqlite3

# 1. Conectar ao banco de dados (ou criar um novo)
conn = sqlite3.connect('exemplo.db')

# 2. Criar um objeto cursor
cursor = conn.cursor()

# 3. Definir o comando SQL para criar a tabela
create_table = """
CREATE TABLE IF NOT EXISTS Produtos (
    id INTEGER PRIMARY KEY,
    nome TEXT NOT NULL,
    preco REAL NOT NULL,
    estoque INTEGER
);
"""

# 4. Executar o comando SQL para criar a tabela
cursor.execute(create_table)
# 5. Confirmar as alterações (commit)
conn.commit()
# 6. Fechar a conexão com o banco de dados
conn.close()
```

1. Importamos o módulo **sqlite3** e conectamos (ou criamos) um banco de dados chamado “exemplo.db”.
2. Criamos um objeto cursor que nos permite executar comandos SQL.
3. Definimos o comando SQL para criar a tabela “Produtos” com campos para “id”, “nome”, “preco” e “estoque”.
4. Executamos o comando SQL usando o cursor.
5. Confirmamos as alterações no banco de dados com **commit()**.
6. Por fim, fechamos a conexão com o banco de dados.

O comando **CREATE TABLE** é um exemplo de DDL (Data Definition Language, ou Linguagem de Definição de Dados), pois possibilita a definição de uma nova estrutura de banco de dados.

Siga em Frente...

CRUD – CREATE, READ, UPDATE, DELETE

Podemos inserir informações (create), ler (read), atualizar (update) e apagar (delete). Os passos necessários para efetuar uma das operações do CRUD são sempre os mesmos: (i) estabelecer a conexão com um banco; (ii) criar um cursor e executar o comando; (iii) gravar a operação; (iv) fechar o cursor e a conexão.

Vamos criar um exemplo no qual haverá a inserção de um novo produto na tabela “Produtos”. Suponhamos que você deseje adicionar um novo produto com nome, preço e quantidade em estoque.

```
import sqlite3
# Conectando ao banco de dados
conn = sqlite3.connect('exemplo.db')
cursor = conn.cursor()
# Dados do novo produto
novo_produto = ('Camiseta', 19.99, 50)
# Comando SQL para inserir o novo produto na tabela
inserir_produto = "INSERT INTO Produtos (nome, preco, estoque) VALUES (?, ?, ?)"
# Executando o comando SQL para inserção
cursor.execute(inserir_produto, novo_produto)
# Confirmando as alterações
conn.commit()
# Fechando a conexão
conn.close()
import sqlite3
```

Confira, a seguir, um exemplo de como você pode recuperar todos os produtos da tabela “Produtos” e exibi-los:

```
# Conectando ao banco de dados
conn = sqlite3.connect('exemplo.db')
cursor = conn.cursor()
# Comando SQL para selecionar todos os produtos
selecionar_produtos = "SELECT * FROM Produtos"
```

```
# Executando o comando SQL
cursor.execute(selecionar_produtos)
# Obtendo todos os registros e exibindo-os
produtos = cursor.fetchall()
for produto in produtos:
    print(produto)
# Fechando a conexão
conn.close()
```

Acompanhe, agora, um exemplo de como atualizar o preço de um produto específico na tabela “Produtos”:

```
import sqlite3
# Conectando ao banco de dados
conn = sqlite3.connect('exemplo.db')
cursor = conn.cursor()
# Novo preço e ID do produto a ser atualizado
novo_preco = 24.99
produto_id = 1 # Suponha que queiramos atualizar o produto com ID 1
# Comando SQL para atualizar o preço do produto
atualizar_preco = "UPDATE Produtos SET preco = ? WHERE id = ?"
# Executando o comando SQL de atualização
cursor.execute(atualizar_preco, (novo_preco, produto_id))
# Confirmando as alterações
conn.commit()
# Fechando a conexão
conn.close()
```

Observe, a seguir, um exemplo de como excluir um produto da tabela “Produtos” com base no seu ID:

```
import sqlite3
# Conectando ao banco de dados
conn = sqlite3.connect('exemplo.db')
cursor = conn.cursor()
# ID do produto a ser excluído
produto_id = 2 # Suponha que queiramos excluir o produto com ID 2
# Comando SQL para excluir o produto
excluir_produto = "DELETE FROM Produtos WHERE id = ?"
# Executando o comando SQL de exclusão
cursor.execute(excluir_produto, (produto_id,))
# Confirmando as alterações
```

```
conn.commit()
# Fechando a conexão
conn.close()
```

É importante lembrar que esses são exemplos simplificados. Em uma aplicação real, talvez você precise adicionar tratamento de erros, validação de dados e outros recursos extras. Por isso é essencial sempre praticar e analisar cada vez mais exemplos para compreender tais conceitos.

Vamos Exercitar?

Vamos pensar no problema apresentado no início desta aula! O objetivo é criar a tabela “Contatos” para armazenar informações de contatos, incluindo nome, e-mail e número de telefone. Vamos ao código!

```
import sqlite3
# CREATE (Criação da tabela e inserção de dados de exemplo)
conn = sqlite3.connect('contatos.db')
cursor = conn.cursor()
cursor.execute("""
    CREATE TABLE IF NOT EXISTS Contatos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nome TEXT,
        email TEXT,
        telefone TEXT
    )
""")
dados_exemplo = [
    ('João', 'joao@email.com', '123-456-7890'),
    ('Maria', 'maria@email.com', '987-654-3210'),
    ('Carlos', 'carlos@email.com', '555-555-5555')
]
cursor.executemany('INSERT INTO Contatos (nome, email, telefone) VALUES (?, ?, ?)',
    dados_exemplo)
conn.commit()
# READ (Leitura e exibição dos contatos)
cursor.execute('SELECT * FROM Contatos')
contatos = cursor.fetchall()
print("Contatos:")
for contato in contatos:
    print(contato)
# UPDATE (Atualização do número de telefone do contato com ID 2)
novo_telefone = '999-999-9999'
```

```
contato_id = 2

cursor.execute('UPDATE Contatos SET telefone = ? WHERE id = ?', (novo_telefone, contato_id))
conn.commit()
# DELETE (Exclusão do contato com ID 1)
contato_id_para_excluir = 1

cursor.execute('DELETE FROM Contatos WHERE id = ?', (contato_id_para_excluir,))
conn.commit()
# Fechando a conexão
conn.close()
```

Nesse código, criamos uma tabela de contatos que permitiu a você praticar as operações CREATE, READ, UPDATE e DELETE em um cenário mais simples.

Espero que tenha gostado da solução! Lembre-se: a prática é importante! Mude alguma parte desse código e diversifique seu conhecimento!

Saiba mais

1. Para conhecer mais detalhes sobre SQL, faça a leitura do livro [Linguagem SQL: fundamentos e práticas](#), cujo link de acesso está disponível a seguir.

CARDOSO, V.; CARDOSO, G. **Linguagem SQL: fundamentos e práticas**. São Paulo: Saraiva, 2013.

2. Como mencionado anteriormente, uma leitura interessante para quem está começando a programar em Python é a do livro [Python 3: conceitos e aplicações: uma abordagem didática](#).

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book.

3. Também encorajo você a conhecer o livro [Banco de dados: projetos e implementação](#), que apresenta conceitos e aplicações de bancos de dados.

MACHADO, F. N. R. **Banco de dados: projeto e implementação**. 3. ed. São Paulo: Érica, 2014.

Referências

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/books/9788536530253>. Acesso em: 21 out. 2023.

CARDOSO, V.; CARDOSO, G. **Linguagem SQL**: fundamentos e práticas. São Paulo: Saraiva, 2013.

LEMBURG, M. PEP 249 - Python database API specification v.2.0. **Python Enhancement Proposals**, 12 abr. 1999. Disponível

em: <https://peps.python.org/pep-0249/#description>. Acesso em: 31 out. 2023.

MACHADO, F. N. R. **Banco de dados**: projeto e implementação. 3. ed. São Paulo: Érica, 2014.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos**: lógica para desenvolvimento de programação de computadores. 29. ed. São Paulo: Érica, 2019.

SQLITE3 – DB-API 2.0 interface for SQLite databases. **Python 3.12.2 Documentation**, [s. d.]. Disponível

em: <https://docs.python.org/3/library/sqlite3.html#module-sqlite3>. Acesso em: 31 out. 2023.

Aula 2

Introdução a Biblioteca Pandas

Introdução à biblioteca pandas

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Dando continuidade à nossa aprendizagem sobre Python, nesta aula vamos conhecer as principais vantagens da biblioteca pandas.

Para uma compreensão correta do conteúdo a ser estudado, o primeiro passo é entender o que é o pandas. Trata-se de uma biblioteca Python de alto desempenho e código aberto projetada para simplificar a manipulação e análise de dados organizados em tabelas e séries temporais. O pandas disponibiliza estruturas de dados poderosas, como DataFrames e Series, que possibilitam aos programadores uma abordagem mais intuitiva e eficiente para lidar com dados tabulares.

Na sequência, devemos entender sobre séries e aprender a criá-las utilizando o pandas. Com base nessa habilidade, teremos uma melhor noção do valor dessa biblioteca para diversos tipos de problemas.

Por fim, descobriremos como ler dados externos usando o pandas. Essa biblioteca contém muitas funcionalidades. Diante disso, para propiciar um estudo mais objetivo, trataremos do caso da leitura dos dados de um site, de forma que o exemplo mostrado incentive a buscar mais conhecimento sobre o pandas.

Para assegurar uma melhor assimilação desse tema, suponha que você esteja gerenciando o cadastro de uma loja cujos diretores precisam de uma orientação sobre o público em que devem investir. Os representantes da loja querem saber a idade média dos seus clientes.

Vamos Começar!

Introdução à biblioteca pandas

pandas é uma poderosa biblioteca de código aberto para a linguagem de programação Python criada para facilitar a manipulação e análise de dados tabulares e séries temporais. Ela fornece estruturas de dados flexíveis e eficientes, como DataFrames e Series, que permitem aos desenvolvedores trabalhar com dados de forma mais intuitiva e produtiva. Algumas características notáveis do pandas incluem:

- 1. DataFrames e Series:** o DataFrame é uma estrutura bidimensional semelhante a uma tabela, enquanto a Series é uma estrutura unidimensional semelhante a uma lista ou matriz. Ambas as estruturas são altamente flexíveis e podem acomodar diversos tipos de dados.
- 2. Manipulação de dados:** o pandas oferece uma ampla gama de funções e métodos para realizar tarefas comuns de manipulação de dados, como filtragem, seleção, ordenação, agrupamento e agregação.

3. **Leitura e escrita de dados:** o pandas suporta a leitura e escrita de dados em vários formatos, incluindo CSV, Excel, SQL, HDF5 e muitos outros, tornando-se uma ferramenta versátil para lidar com dados de diferentes fontes.
4. **Tratamento de dados ausentes:** a biblioteca simplifica o tratamento de dados faltantes, permitindo que os desenvolvedores preencham ou removam valores ausentes de forma eficaz.
5. **Visualização de dados:** embora o pandas seja mais conhecido por sua capacidade de manipular dados, também pode ser integrado a outras bibliotecas de visualização, como Matplotlib e Seaborn, para criar gráficos e visualizações informativas.
6. **Integração com NumPy:** o pandas é construído sobre a biblioteca NumPy, o que significa que você pode facilmente combinar as capacidades de NumPy para cálculos numéricos com as funcionalidades do pandas para manipulação de dados.
7. **Comunidade ativa:** o pandas tem uma comunidade de usuários e desenvolvedores ativa, o que resulta em suporte contínuo e atualizações regulares.

O pandas é amplamente utilizado em análise de dados, ciência de dados e engenharia de dados. Ele oferece uma maneira eficiente e amigável de lidar com dados, fato que o torna uma escolha popular para profissionais que trabalham com informações estruturadas.

Series

Para criar um objeto do tipo Series no pandas, utilizamos o método `Series()` com vários parâmetros opcionais. O principal parâmetro é "data", que pode conter um único valor, uma lista de valores ou um dicionário. Outros parâmetros, como "index", "dtype" e "name", têm valores-padrão predefinidos, tornando sua especificação opcional. A documentação oficial do pandas fornece detalhes completos sobre esses parâmetros. Saiba mais em: [pandas](#).

Exemplo 1: criar uma Series a partir de uma lista

```
import pandas as pd

# Criando uma lista de valores
data = [10, 20, 30, 40, 50]

# Criando uma Series a partir da lista
series1 = pd.Series(data)

print(series1)

#resultado
```

```
0  10
1  20
2  30
3  40
4  50
dtype: int64
```

Exemplo 2: criar uma Series a partir de um dicionário

```
import pandas as pd

# Criando um dicionário com pares chave-valor
data = {'A': 100, 'B': 200, 'C': 300, 'D': 400, 'E': 500}

# Criando uma Series a partir do dicionário
series2 = pd.Series(data)

print(series2)

#resultado
A    100
B    200
C    300
D    400
E    500
dtype: int64
```

No primeiro exemplo, uma Series é criada a partir de uma lista de valores. Já no segundo, uma Series é criada a partir de um dicionário, de modo que as chaves se tornam os índices da Series e os valores são os dados correspondentes.

Siga em Frente...

Leitura de dados estruturados com a biblioteca pandas

Um recurso poderoso no pandas é a capacidade de ler dados estruturados e armazená-los em um DataFrame. A biblioteca oferece vários métodos de leitura de dados, identificados pelo padrão “read”, como `pandas.read_XXXXX()`. Cada um desses métodos é projetado para ler diferentes tipos de fontes de dados.

Para exemplificar, vamos explorar o método `pandas.read_html()`, que é utilizado para extrair tabelas de uma página da web. Esse método procura automaticamente por elementos HTML `<table>` na estrutura da página e retorna uma lista de DataFrames que correspondem às tabelas encontradas. Os parâmetros, como “io”, podem ser configurados para especificar a URL da página a ser lida, e outros parâmetros adicionais podem ser ajustados para lidar com formatação e tratamento de dados. Isso torna o pandas uma ferramenta versátil para a aquisição de dados de fontes externas, como páginas da web.

Na URL [Lista de bancos com falha](https://www.fdic.gov/resources/resolutions/bank-failures/failed-bank-list/) encontra-se uma tabela com bancos norte-americanos que faliram desde 1º de outubro de 2000. Nesse caso, cada linha representa um banco.

```
import pandas as pd
url = 'https://www.fdic.gov/resources/resolutions/bank-failures/failed-bank-list/'
dfs = pd.read_html(url)

print(type(dfs))
print(len(dfs))
#resultado
<class 'list'>
1
Portanto, temos nosso DataFrame com uma lista que contém apenas uma tabela.
df_bancos = dfs[0]

print(df_bancos.shape)
print(df_bancos.dtypes)

df_bancos.head()
#resultado
(567, 7)
Bank NameBank      object
CityCity           object
StateSt            object
CertCert           int64
Acquiring InstitutionAI  object
Closing DateClosing  object
FundFund           int64
dtype: object
```

	Bank NameBank	CityCity	StateSt	CertCert	Acquiring InstitutionAI	Closing DateClosing	FundFund
0	Heartland Tri-State Bank	Elkhart	KS	25851	Dream First Bank, N.A.	July 28, 2023	10544
1	First Republic Bank	San Francisco	CA	59017	JPMorgan Chase Bank, N.A.	May 1, 2023	10543
2	Signature Bank	New York	NY	57053	Flagstar Bank, N.A.	March 12, 2023	10540
3	Silicon Valley Bank	Santa Clara	CA	24735	First-Citizens Bank & Trust Company	March 10, 2023	10539
4	Almena State Bank	Almena	KS	15426	Equity Bank	October 23, 2020	10538

Figura 1 | Cinco primeiras linhas do DataFrame. Fonte: elaborada pelo autor.

Nesse exemplo, trouxemos o tipo de cada variável existente no DataFrame, além das cinco primeiras linhas como o comando `head()`. Com isso, mostramos quão rica é a biblioteca pandas e suas diversas aplicações.

Vamos Exercitar?

Agora, vamos colocar em prática o que aprendemos nesta aula para resolver nosso problema inicial. Podemos usar a biblioteca pandas para descobrir em qual público a loja deve investir. Para tanto, devemos calcular a média de idade dos clientes.

```
import pandas as pd

# Criar um dicionário com nomes e idades
dados = {
    'Nome': ['Alice', 'Bob', 'Carol', 'David', 'Eve'],
    'Idade': [25, 30, 22, 35, 28]
}

# Criar uma série a partir do dicionário
serie_idades = pd.Series(dados['Idade'], index=dados['Nome'])

# Exibir a série de idades
print("Série de Idades:")
print(serie_idades)

# Calcular a média das idades
media_idades = serie_idades.mean()
print("\nMédia de Idades:", media_idades)

#resultado
Série de Idades:
Alice    25
Bob      30
Carol    22
David    35
Eve      28
dtype: int64
```

Média de Idades: 28.0

Começamos esse exercício criando um dicionário chamado **dados**, que contém os nomes e idades dos clientes. Em seguida, usamos o pandas para criar uma série chamada **serie_idades** a partir desse dicionário, de forma que os nomes são definidos como o índice da série.

Após criar a série, calculamos a média das idades usando o método `mean()` e a exibimos na saída.

A saída do código mostra a série de idades e a média das idades do grupo. Esse é um exemplo simples de como você pode usar o pandas para realizar cálculos em dados estruturados.

Gostou dessa solução? Espero que sim! Já estamos avançando bastante em nossa trajetória de estudos, desta vez utilizando o pandas, uma biblioteca do Python. Faça mudanças no código e pratique!

Saiba mais

1. Para entender mais detalhes sobre o uso do pandas, sugiro que você visite a seguinte página: [pandas](#). O site descreve a história e as funcionalidades dessa biblioteca.
2. Agora, para aprender mais sobre DataFrame, acesse o endereço a seguir: [pandas](#).
3. Por fim, para exercitar os assuntos estudados nesta etapa de aprendizagem, faça a leitura do livro [Python 3: conceitos e aplicações: uma abordagem didática](#).

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book.

Referências

ABOUT pandas. **pandas**, [s. d.]. Disponível em: <https://pandas.pydata.org/about/>. Acesso em: 31 out. 2023.

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9788536530253>. Acesso em: 21 out. 2023.

FAILED Bank List. The Federal Deposit Insurance Corporation, 3 nov. 2023. Disponível em: <https://www.fdic.gov/resources/resolutions/bank-failures/failed-bank-list/>. Acesso em: 31 jan. 2024.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos**: lógica para desenvolvimento de programação de computadores. 29. ed. São Paulo: Érica, 2019.

PANDAS.DATAFRAME. **pandas**, [s. d.]. Disponível em: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.html>. Acesso em: 31 out. 2023.

Aula 3

Introdução a manipulação de dados em Panda

Introdução à manipulação de dados em panda

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Para aprofundar nossos conhecimentos sobre o mundo da linguagem Python, devemos aprender a utilizar cada vez mais ferramentas que facilitem a preparação de nossos códigos. Já descobrimos que existem inúmeras bibliotecas que nos ajudam em diversas situações, dentre as quais se destaca a biblioteca pandas, que fornece muitas funcionalidades para o trabalho com dados.

Sendo assim, devemos conhecer os métodos para leitura e escrita da biblioteca pandas. Atualmente, os dados provêm de diversas fontes e de diferentes formatos de arquivo. Diante disso, é necessário saber lidar com cada um desses dados.

Nesse sentido, devemos entender como trabalhar com tais dados e, mais do que isso, como transformá-los, visto que a maioria deles não é passível de uso caso não passe por uma manipulação prévia.

Por fim, é preciso obter informações sobre os dados. Para tanto, conheceremos duas ferramentas: loc. e testes booleanos.

Suponha, agora, que você trabalhe em uma loja que vende itens variados. Por conta de um erro no sistema de venda, o valor unitário dos itens vendidos não é exibido, e existem algumas duplicações de linhas. Você precisa mostrar itens com valores acima de R\$50,00 para o planejamento da empresa em uma ação de marketing. Vamos, juntos, resolver esse caso?

Vamos Começar!

Métodos para leitura e escrita da biblioteca pandas

A biblioteca pandas tem como principal propósito a manipulação de dados estruturados, como aqueles organizados em tabelas com linhas e colunas.

Esses dados podem ser provenientes de diversas fontes, como arquivos, páginas web, APIs, outros softwares, serviços de armazenamento em nuvem e bancos de dados. A biblioteca oferece uma variedade de métodos que permitem a leitura e o carregamento desses dados em estruturas chamadas DataFrames.

Os métodos de leitura de dados estruturados no pandas têm em comum o prefixo “pd.read_XXXX”, sendo “pd” um alias frequentemente utilizado ao importar a biblioteca e “XXX” a parte restante da sintaxe específica de cada método. Além da leitura, o pandas oferece diversos métodos para escrever os dados contidos em um DataFrame em arquivos, bancos de dados ou até mesmo para a área de transferência do sistema operacional. Isso torna o pandas uma ferramenta versátil para lidar com dados estruturados, independentemente de sua origem.

O Quadro 1, a seguir, mostra os métodos de leitura e escrita para os diferentes tipos de dados:

Tipo de Dado	Descrição do Dado	Método para Leitura	Método para Escrita
Texto	CSV	read_csv	to_csv
Texto	Fixe-width texto file	read_fwf	

Texto	JSON	read_json	to_json
Texto	HTML	read_html	to_html
Texto	Latex		styler.to_latex
Texto	XML	read_xml	to_xml
Texto	Local Clipboard	read_clipboard	to_clipboard
Binário	MS Excel	read_excel	to_excel
Binário	OpenDocument	read_excel	
Binário	HDF5 Format	read_hdf	to_hdf
Binário	Feather Fomart	read_feather	to_feather
Binário	Parquet Format	read_parquet	to_parquet
Binário	ORC Format	read_orc	
Binário	MsgPack	read_msgpack	to_msgpack
Binário	Stata	read_stata	to_stata
Binário	SAS	read_sas	
Binário	SPSS	read_spss	
Binário	Python Pickle Format	read_pickle	to_picke
SQL	SQL	read_sql	to_sql
SQL	Google BigQuery	read_gbq	to_gbq

Quadro 1 | Métodos de leitura e escrita. Fonte: adaptado de pandas ([s. d.]a).

Captura e transformação dos dados

A etapa de captura e transformação/padronização dos dados é uma parte crucial no processo de análise de dados e modelagem de machine learning, por exemplo. Nessa fase, você coleta os dados brutos de várias fontes, como arquivos CSV, bancos de dados, APIs, e os prepara para uma análise posterior. O pandas é uma biblioteca Python muito útil para realizar essas tarefas, pois fornece estruturas de dados flexíveis e ferramentas poderosas para manipular e transformar dados.

Vamos analisar um exemplo:

```
import pandas as pd
```

```
df_selic = pd.read_json("https://api.bcb.gov.br/dados/serie/bcdata.sgs.11/dados?
formato=json")

print(df_selic.info())
#resultado
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9379 entries, 0 to 9378
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0  data    9379 non-null    object
1  valor   9379 non-null    float64
dtypes: float64(1), object(1)
memory usage: 146.7+ KB
None
```

O primeiro passo para o desenvolvimento desse processo é importar o pandas. Em seguida, utilizamos o método para ler um arquivo JSON. Por fim, pedimos informações sobre esse DataFrame: existem 9.379 registros, 2 colunas, os índices são numéricos e variam de 0 a 9.378; não existem linhas faltantes, pois existem 9.379 registros não nulos; os dados são do tipo "object", ou são todos strings, ou há uma mistura desses tipos.

Já conhecemos nosso DataFrame. Agora, vamos verificar a duplicidade de linhas (um passo muito importante) utilizando a função `drop_duplicates()`. No nosso exemplo, usaremos: `df_selic.drop_duplicates(keep='last', inplace=True)`, que mantém o último registro (`keep='last'`) e, a partir do parâmetro `inplace=True`, faz com que a transformação seja salva do DataFrame. Na prática, estamos sobrescrevendo o objeto na memória. Nesse caso, não existem linhas duplicadas.

Outra ação que podemos efetuar é criar uma nova coluna no DataFrame. Para isso, a sintaxe é simples: `df['nova_coluna'] = dado`. No nosso caso, vamos inserir duas colunas, uma com a data da extração dos dados e outra com o responsável pela extração.

```
from datetime import date
from datetime import datetime as dt

data_extracao = date.today()

df_selic['data_extracao'] = data_extracao
df_selic['responsavel'] = "Autor"

print(df_selic.info())
df_selic.head()
```

	data	valor	data_extracao	responsavel
0	04/06/1986	0.065041	2023-11-02	Autor
1	05/06/1986	0.067397	2023-11-02	Autor
2	06/06/1986	0.066740	2023-11-02	Autor
3	09/06/1986	0.068247	2023-11-02	Autor
4	10/06/1986	0.067041	2023-11-02	Autor

Figura 1 | Dados da extração. Fonte: elaborada pelo autor.

Usamos os módulos datetime, classe date e o método today(). Ao criar a coluna, a biblioteca pandas “entende” que se deve colocar o valor em todas as linhas, isto é, tanto a data da extração quanto o responsável. A manipulação/transformação dos dados muda conforme as especificidades da situação e do problema envolvidos.

Siga em Frente...

Extração de informações

Depois de saber coletar os dados e transformá-los, devemos passar para o próximo passo, que é extrair informação deles. Nessa etapa, precisamos conhecer o que estamos procurando para tentar encontrar esse elemento nos dados. Existem inúmeras ferramentas e maneiras pelas

quais podemos fazer isso, como por meio de filtros utilizando loc., filtros utilizando testes booleanos, entre outras medidas.

Veja, a seguir, como utilizar loc.:

```
df_selic.loc[0]
#resultado
data 04/06/1986 valor 0.065041 data_extracao 2023-11-02 responsavel Autor Name: 0, dtype: object

df_selic.loc[[0,20,70]]
#resultado
   Data      valor  data_extracao responsavel
0 - 04/06/1986 - 0.065041 - 2023-11-02 - Autor
20 - 02/07/1986 - 0.068301 - 2023-11-02 - Autor
70 - 10/09/1986 - 0.131315 - 2023-11-02 - Autor
```

Há muitas maneiras de usar loc. de forma eficaz. Para saber mais detalhes sobre esse assunto, acesse: [pandas](#).

Confira, agora, um exemplo no qual se utiliza o teste booleano:

```
teste = df_selic['valor'] < 0.01
print(type(teste))
#resultado
<class 'pandas.core.series.Series'>
```

Como resultado, esse teste traz duas saídas para cada valor: true ou false. Testes booleanos são de grande importância para diversas situações.

Vamos Exercitar?

Agora que já aprendemos mais detalhes sobre o pandas e algumas de suas funcionalidades, vamos resolver o problema apresentado no início desta aula. Para isso, traremos dados fictícios, os quais podem ser alterados para que você pratique cada vez mais.

```
import pandas as pd

# Criando um DataFrame com 5 linhas de dados
data = {
```

```
'nome': ['Produto A', 'Produto B', 'Produto C', 'Produto A', 'Produto E'],
'quantidade de itens comprados': [3, 1, 4, 3, 2],
'tipo de item': ['Eletrônico', 'Vestuário', 'Alimento', 'Eletrônico', 'Alimento'],
'receita total': [120, 80, 60, 120, 90]
}
df = pd.DataFrame(data)

# Duplicando uma linha
df.drop_duplicates(keep='last', inplace=True)

# Calculando a coluna 'preço do item'
df['preço do item'] = df['receita total'] / df['quantidade de itens comprados']

# Selecionando preço do item acima de 50 reais
itens_acima_de_50 = df[df['preço do item'] > 50]

print("Itens acima de 50 reais:")
print(itens_acima_de_50)
#resultado
Itens acima de 50 reais:
   nome  quantidade de itens comprados  tipo de item  receita total
1  Produto B             1          Vestuário             80

   preço do item
1          80.0
```

Criamos um DataFrame com dados fictícios. Observe que temos duas linhas duplicadas e, depois de excluir uma delas, ficamos com quatro itens, sendo que somente o produto B do vestuário tem valor acima de R\$50,00.

Faça mudanças e descubra outras possibilidades para solucionar o problema. Lembre-se de que a prática é importante para aprender cada vez mais.

Saiba mais

1. O livro [Introdução à computação usando Python: um foco no desenvolvimento de aplicações](#) apresenta uma introdução à programação, ao desenvolvimento de aplicações de computador e à ciência da computação. Logo, para você, que está iniciando seu aprendizado em Python, a leitura desse texto é muito relevante. Para acessar o material sugerido, clique no link a seguir.

PERKOVIC, L. **Introdução à computação usando Python**: um foco no desenvolvimento de aplicações. Rio de Janeiro: LTC, 2016.

2. Uma leitura interessante para quem está começando a programar em Python é a do livro [*Começando a programar em Python para leigos*](#).

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book.

3. Outra dica para que você aprofunde seu entendimento sobre a aplicação da linguagem Python em situações reais é a leitura do artigo [*Aplicação de técnicas de aprendizado de máquina e estatística na previsão da demanda de biocombustíveis*](#).

PAULA, J. de S. *et al.* Aplicação de técnicas de aprendizado de máquina e estatística na previsão da demanda de biocombustíveis. **Revista de Gestão e Secretariado**, São Paulo, v. 13, n. 4, ed. esp., p. 2559-2572, 2022.

Referências

GOOGLE COLAB. **Página inicial**, [s. d.]. Disponível em: <https://colab.research.google.com/>. Acesso em: 12 out. 2023.

IO tools (text, CSV, HDF5, ...). **pandas**, [s. d.]a. Disponível em: https://pandas.pydata.org/pandas-docs/stable/user_guide/io.html. Acesso em: 2 nov. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos**: lógica para desenvolvimento de programação de computadores. 29. ed. São Paulo: Érica, 2019.

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555202298>. Acesso em: 12 out. 2023.

PANDAS.DATAFRAME.LOC. **pandas**, [s. d.]b. Disponível em: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.loc.html>. Acesso em: 2 nov. 2023.

PAULA, J. de S. *et al.* Aplicação de técnicas de aprendizado de máquina e estatística na previsão da demanda de biocombustíveis. **Revista de Gestão e Secretariado**, São Paulo, v. 13, n. 4, ed. esp., p. 2559-2572, 2022. Disponível em: <https://ojs.revistagesec.org.br/secretariado/article/view/1488/708>. Acesso em: 12 out. 2023.

PERKOVIC, L. **Introdução à computação usando Python**: um foco no desenvolvimento de aplicações. Rio de Janeiro: LTC, 2016.

Aula 4

Visualização de Dados em Python

Visualização de dados em Python

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Que a linguagem Python tem muitas qualidades você já sabe, agora vamos aprender a utilizar algumas bibliotecas para visualizar dados de diferentes maneiras.

Uma biblioteca já mencionada no decorrer desta disciplina é a Matplotlib, que desempenha um papel fundamental, funcionando como uma pedra angular na criação de gráficos em Python. Trata-se de uma escolha universalmente aceita em projetos de visualização de dados.

Outra biblioteca bastante renomada é o pandas, que, além de suas funcionalidades amplamente conhecidas, disponibiliza recursos de visualização gráfica.

Por fim, examinaremos a biblioteca Seaborn, que é uma extensão da base do Matplotlib, destacando-se como uma ferramenta especializada para a criação de gráficos de alta qualidade

em Python.

Suponha que você precise responder em qual período os clientes gastam mais em um restaurante e verificar se esse mesmo período é aquele em que eles dão mais gorjetas. Como poderíamos representar as respostas graficamente?

Vamos Começar!

Matplotlib

A biblioteca Matplotlib exerce uma função central na criação de gráficos em Python, sendo amplamente adotada em projetos de visualização de dados. John Hunter é o criador e uma figura-chave para o desenvolvimento dessa biblioteca, que surge como uma alternativa ao uso de ferramentas como gnuplot e MATLAB na comunidade científica. Anteriormente, os cientistas tinham que gerar gráficos em outros softwares após extrair os resultados de suas análises, fato que tornava o processo incômodo. Assim, a biblioteca Matplotlib emergiu como uma solução eficiente para criar visualizações em Python.

A instalação do Matplotlib pode ser facilmente realizada com o comando “pip install matplotlib”. Em ambientes como Anaconda e Google Colab, essa biblioteca já está prontamente disponível. O módulo “pyplot” é uma parte essencial do Matplotlib, pois disponibiliza funções que facilitam a criação e personalização de gráficos. Duas sintaxes comuns para importar o Matplotlib com o apelido “plt” são: “import matplotlib.pyplot as plt” e “from matplotlib import pyplot as plt”.

Os gráficos desempenham o papel de narradores visuais, contando histórias por meio dos dados. Para começar nossa jornada de visualização, vamos usar algo criativo e abstrato. Geraremos duas listas de valores inteiros aleatórios usando o módulo “random” e criaremos um gráfico de linhas com o Matplotlib.

```
import matplotlib.pyplot as plt
import random

dados1 = random.sample(range(100), k=20)
dados2 = random.sample(range(100), k=20)

plt.plot(dados1, dados2) # pyplot gerencia a figura e o eixo
```

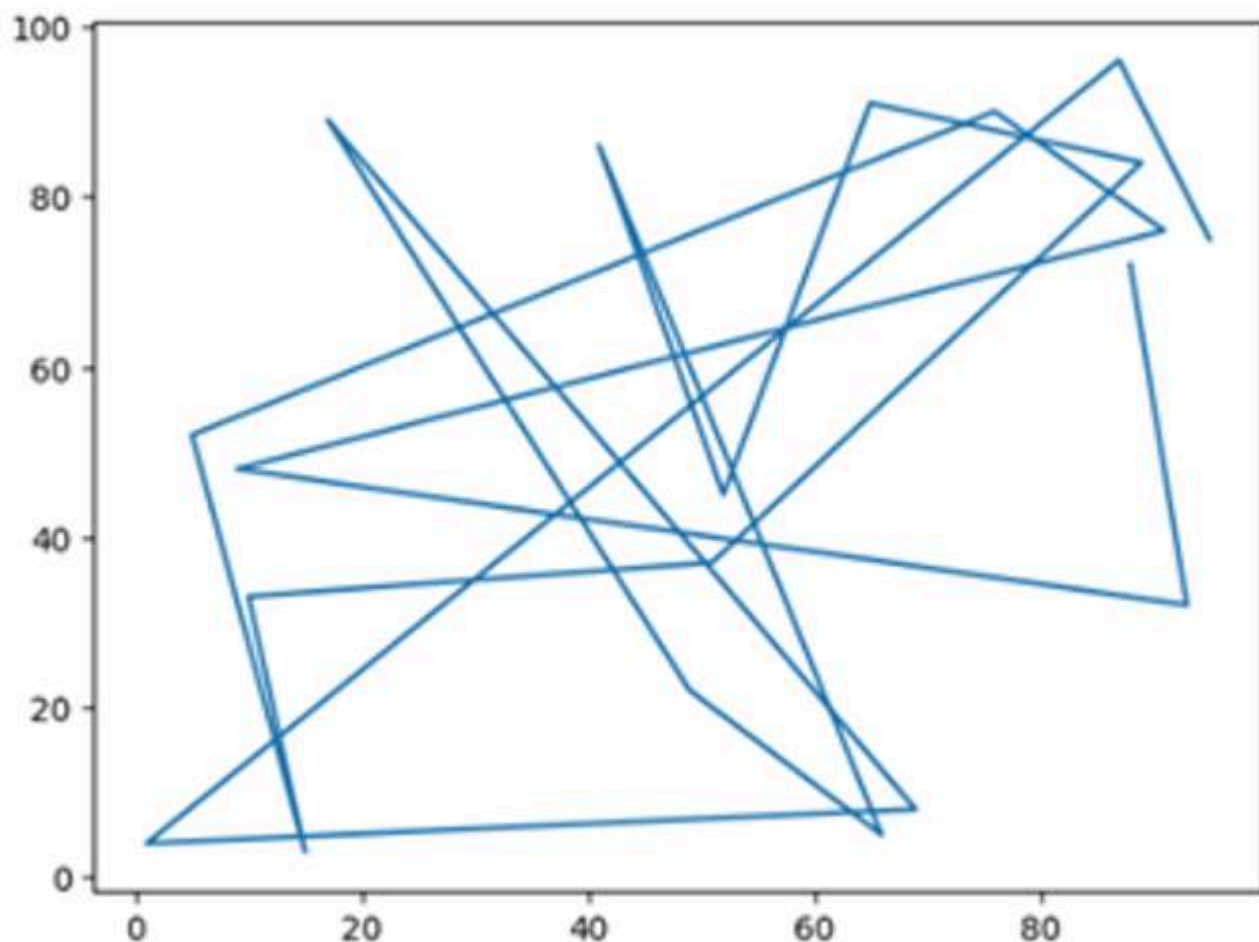


Figura 1 | Gráfico. Fonte: elaborada pelo autor.

Existem duas formas de criar o gráfico:

1. O pyplot cria e gerencia automaticamente figuras e eixos, e usa as funções do pyplot para plotagem.
2. Criar explicitamente figuras e eixos, e chamar métodos sobre eles (o “estilo orientado a objetos (OO)”).

No gráfico criado, utilizamos a opção 1, ou seja, foi o próprio módulo que criou o ambiente da figura e do eixo. Como já aprendemos ao estudar essa biblioteca, ela é de extrema importância para diferentes áreas.

Biblioteca pandas

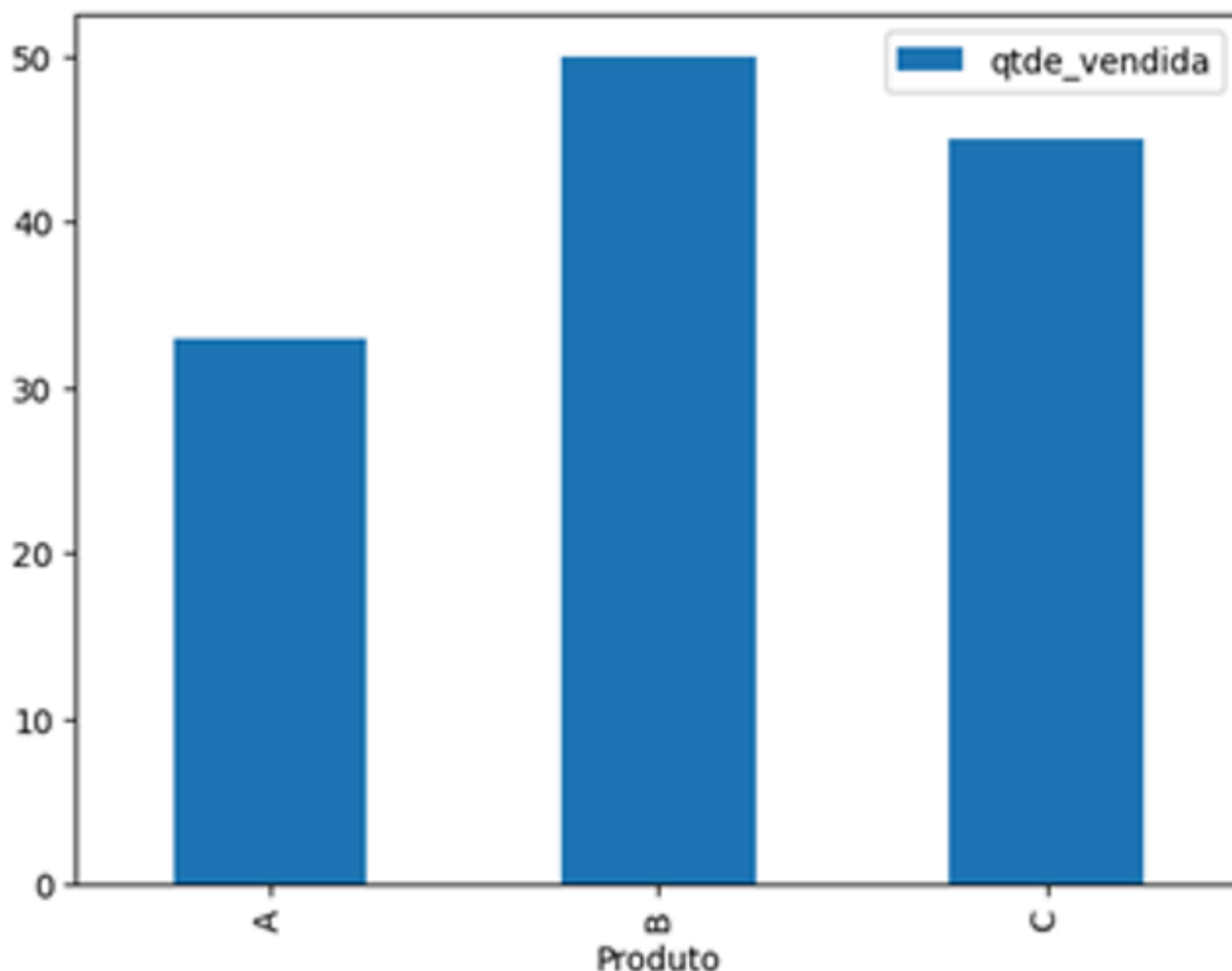
Já conhecemos muitas funcionalidades da biblioteca pandas; uma delas diz respeito à visualização gráfica. As principais estruturas de dados da biblioteca pandas (Series e

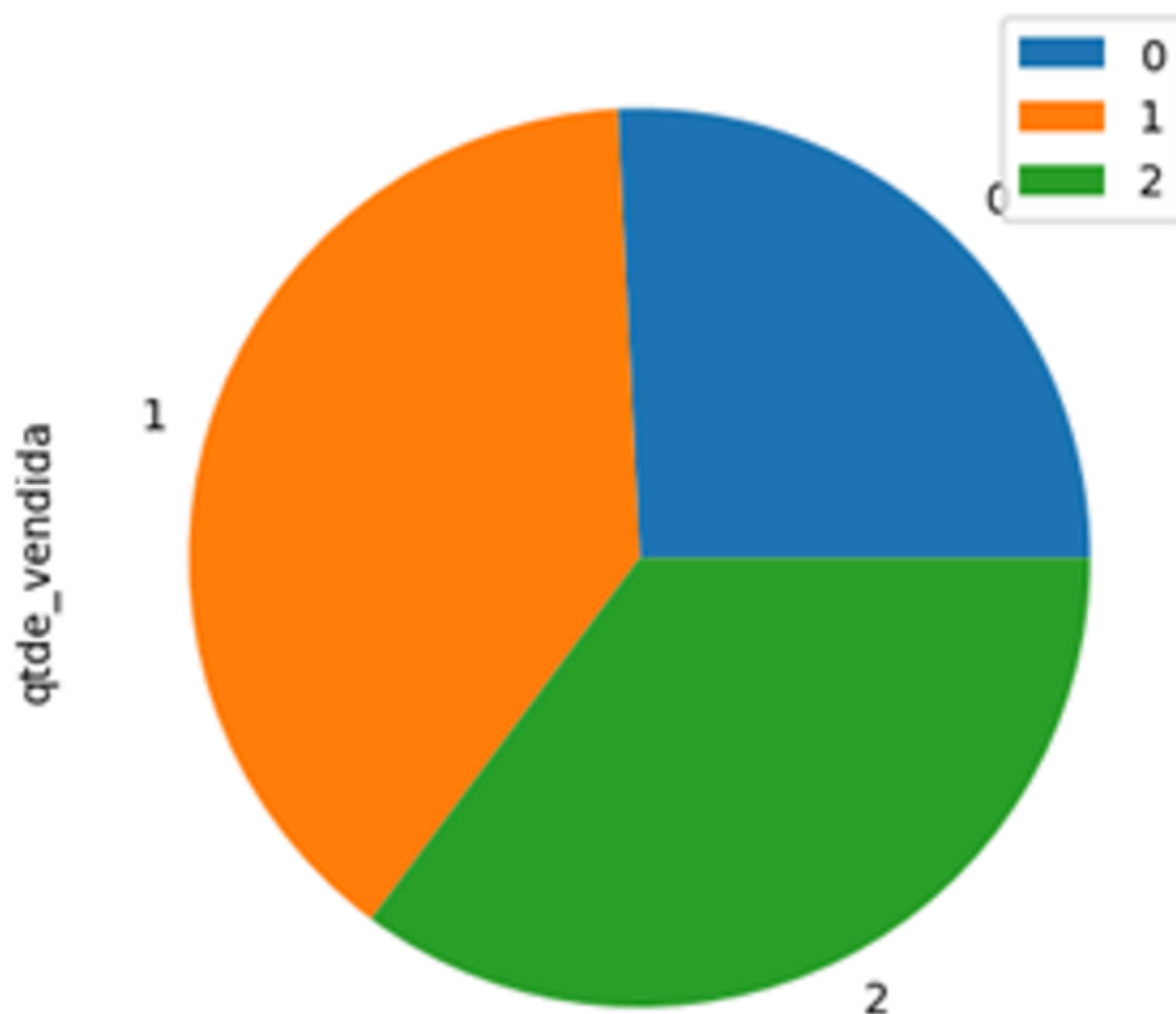
DataFrame) possuem o método `plot()`, construído com base no Matplotlib e que permite criar gráficos a partir dos dados nas estruturas.

Confira um exemplo:

```
import pandas as pd

dados = {
    'Produto': ['A', 'B', 'C'],
    'qtde_vendida': [33, 50, 45]
}
df = pd.DataFrame(dados)
df.plot(x='Produto', y='qtde_vendida', kind='bar')
df.plot(x='Produto', y='qtde_vendida', kind='pie')
df.plot(x='Produto', y='qtde_vendida', kind='line')
```





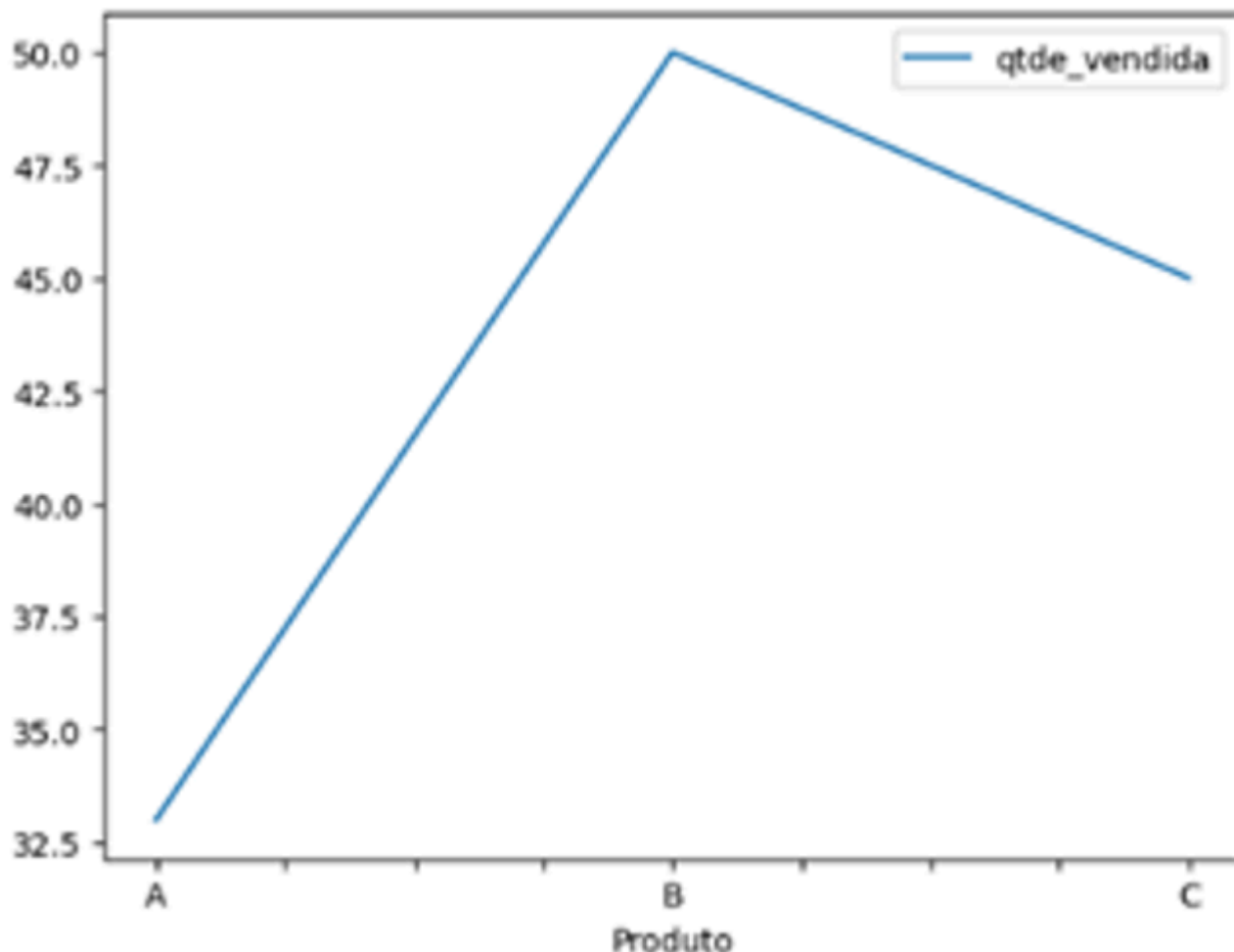


Figura 2 | Gráficos criados. Fonte: elaborada pelo autor.

Existem muitas visualizações possíveis. No exemplo anterior, utilizamos os gráficos de barras, de pizza e de linhas. No endereço [pandas](#) você encontra a lista com todos os tipos de gráficos que podem ser construídos com o método `plot()` da biblioteca.

Siga em Frente...

Biblioteca Seaborn

O Seaborn, uma biblioteca Python construída sobre a base do Matplotlib, destaca-se na criação de gráficos de forma especializada. Você pode usar essa biblioteca importando-a em seus projetos da seguinte forma: `import seaborn as sns`. Uma característica notável do Seaborn é seu repositório de conjuntos de dados prontos para uso, o que facilita a exploração das funcionalidades. Você pode acessar esses conjuntos de dados em [mwaskom](#). Para ilustrar,

vamos carregar dados sobre gorjetas (tips) e utilizá-los em nosso estudo. O Seaborn simplifica a criação de gráficos e as análises de dados, mostrando-se uma ferramenta valiosa para a visualização de informações.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="whitegrid") # opções: darkgrid, whitegrid, dark, white, ticks

df_tips = sns.load_dataset('tips')

fig, ax = plt.subplots(1, 3, figsize=(15, 5))

sns.barplot(data=df_tips, x='sex', y='total_bill', ax=ax[0])
sns.barplot(data=df_tips, x='sex', y='total_bill', ax=ax[1], estimator=sum)
sns.barplot(data=df_tips, x='sex', y='total_bill', ax=ax[2], estimator=len)
```

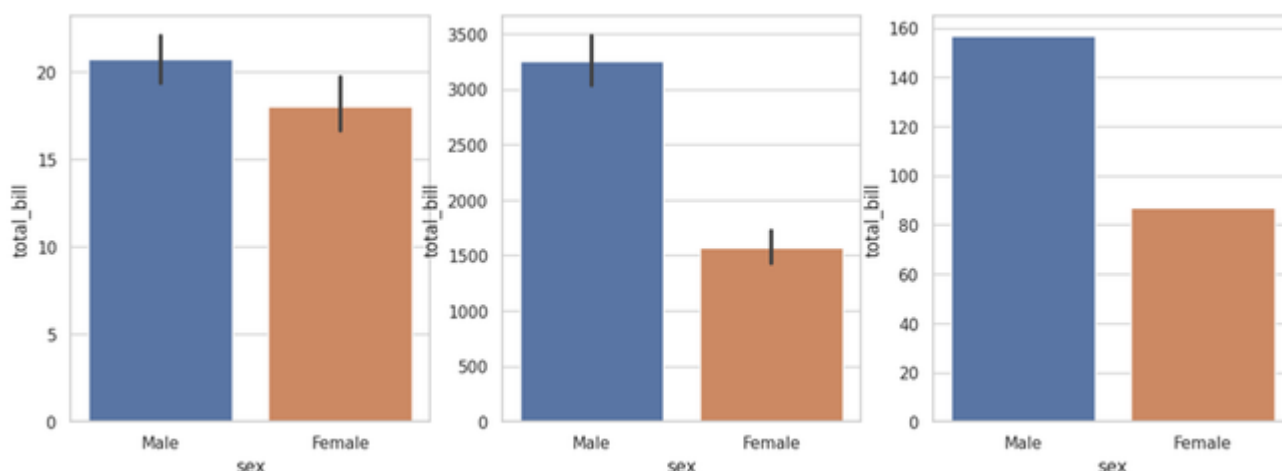


Figura 3 | Gráficos utilizando barplot(). Fonte: elaborada pelo autor.

A escolha entre a função “barplot()” do Seaborn e as funcionalidades de gráficos de barras no pandas dependerá das necessidades específicas da análise de dados. O motivo que leva a optar pelo “barplot()” muitas vezes se baseia nos parâmetros adicionais e na flexibilidade que ele oferece. Vamos dar destaque ao parâmetro “estimator”, que, por padrão, calcula a média.

A função “barplot()” do Seaborn apresenta uma variedade de opções estatísticas, permitindo que os cientistas de dados escolham a métrica que melhor se ajuste aos seus objetivos. Por exemplo, você pode calcular a soma, a contagem ou até mesmo outras métricas personalizadas. Isso é particularmente útil quando você deseja exibir informações diferentes nas barras, como a quantidade (len) ou a soma (sum) dos valores, em vez da média.

Em contraste, o pandas concede funcionalidades de gráficos de barras mais básicas, que geralmente se concentram na representação da média dos dados. Portanto, a escolha entre as duas abordagens dependerá da necessidade de personalização e da complexidade da análise estatística que você deseja realizar. O Seaborn fornece mais controle e opções para criar gráficos de barras que atendam precisamente às demandas de seu projeto.

Para ilustrar como o parâmetro afeta a construção do gráfico, observe a Figura 1, apresentada anteriormente. Nesse caso, usamos o Matplotlib para construir uma figura e um eixo com três posições. No primeiro gráfico, utilizou-se o padrão, isto é, a média; no segundo gráfico, a função soma; e, no terceiro, a função len.

Ao observar os resultados dos gráficos, podemos perceber as diferenças significativas entre eles. O primeiro gráfico nos fornece uma ideia de que o valor médio da conta entre homens e mulheres é semelhante, com uma ligeira vantagem para os homens. No entanto, o segundo gráfico nos dá a impressão de que os homens gastam muito mais em média. Mas será que isso é realmente verdade?

A resposta está no contexto dos dados. É importante verificar se a quantidade de homens na base de dados é significativamente maior do que a de mulheres, o que pode influenciar a soma total das contas. O terceiro gráfico esclarece essa questão, revelando o número de homens e mulheres na base de dados. Com essa informação, podemos avaliar se a diferença na soma total das contas entre os grupos ocorre por causa da disparidade na quantidade de observações. Portanto, a interpretação correta dos gráficos requer uma análise contextual, para evitar conclusões precipitadas.

Vamos Exercitar?

Vamos resolver o problema descrito no início desta aula. Para isso, utilizaremos o DataSet sobre gorjeta apresentado nesta etapa de aprendizagem.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="whitegrid")

df = sns.load_dataset('tips')

plt.figure(figsize=(8, 5))
sns.barplot(x='time', y='total_bill', data=df, estimator=sum, ci=None, palette="Set2")
plt.xlabel('Período (Time)')
plt.ylabel('Total de Gastos')
plt.title('Total de Gastos por Período (Almoço ou Jantar)')
```

```
plt.show()
```

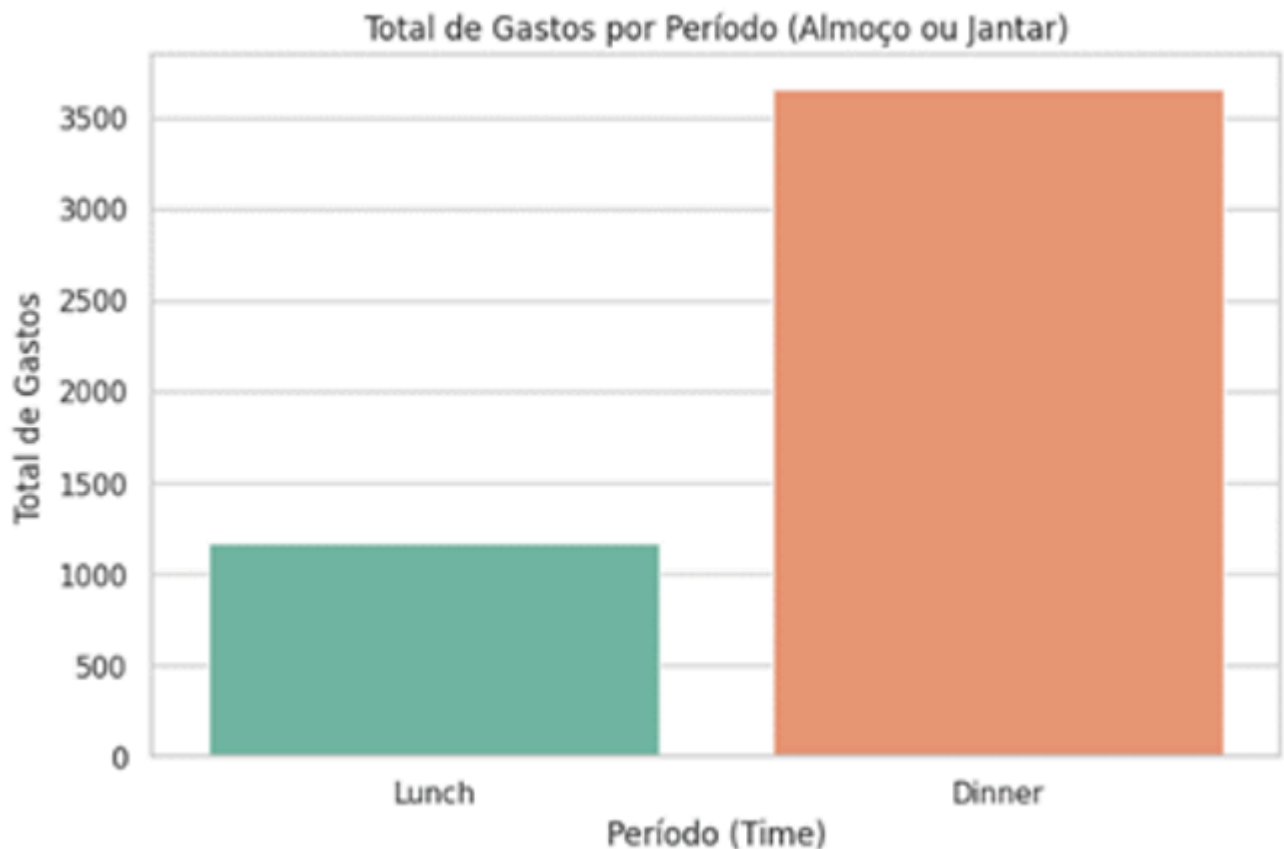


Figura 4 | Total de gastos por período (almoço ou janta). Fonte: elaborada pelo autor.

Já conseguimos verificar que o gasto no jantar é bem superior ao gasto no almoço. Mas quantos clientes almoçaram e quantas clientes jantaram? Vamos investigar o gasto médio por período.

```
plt.figure(figsize=(8, 5))
sns.barplot(x='time', y='total_bill', data=df)#, #estimator=sum, ci=None, palette="Set2")
plt.xlabel('Período (Time)')
plt.ylabel('Média de Gastos')
plt.title('Média de Gastos por Período (Almoço ou Jantar)')
plt.show()
```

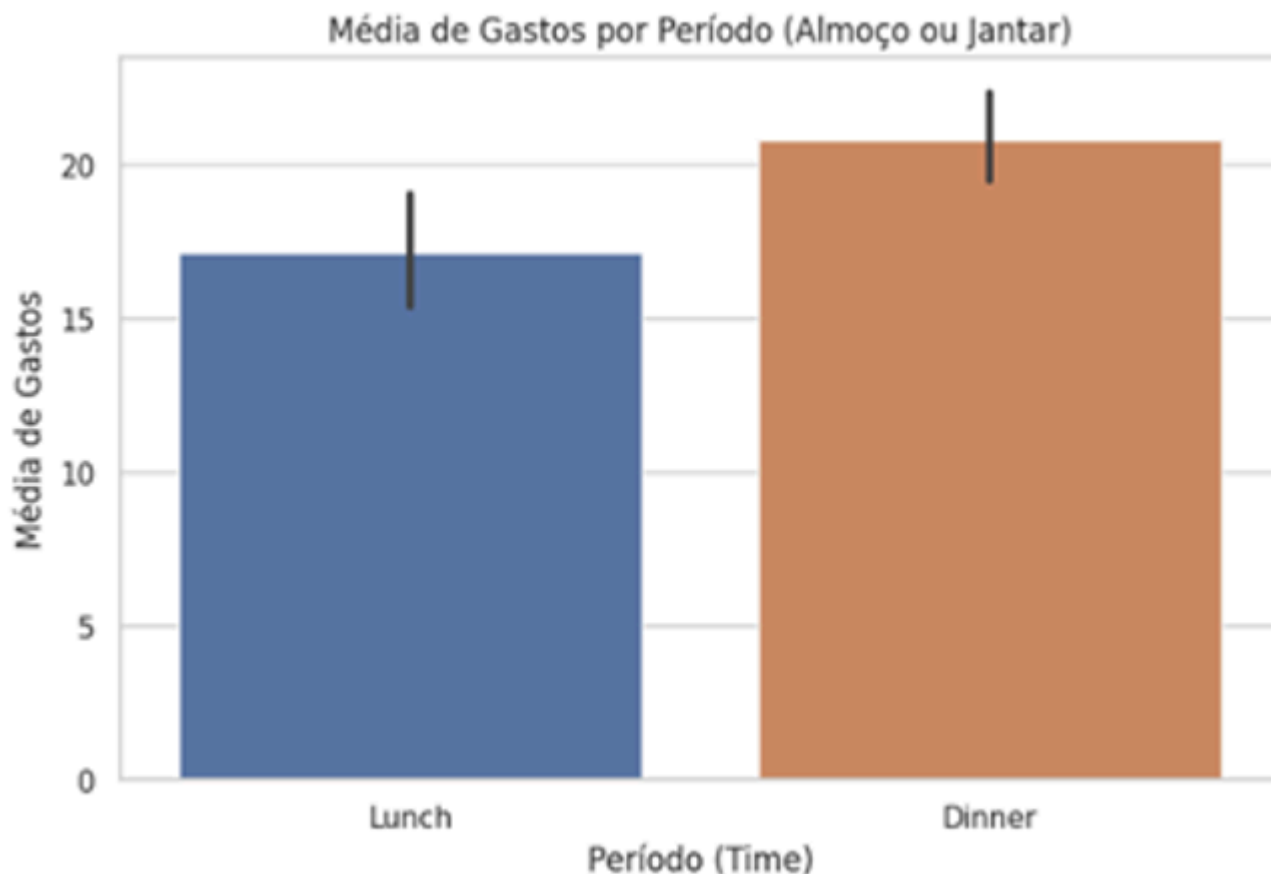


Figura 5 | Média de gastos por período (almoço ou janta). Fonte: elaborada pelo autor.

Entendemos que a média gasta no jantar também é superior à do almoço. Logo, esse é o principal período do restaurante em relação a faturamento.

Por fim, vamos verificar a média de gorjeta por período.

```
# Crie um gráfico de barras com o Seaborn para mostrar a média de gorjetas por período
plt.figure(figsize=(8, 5))
sns.barplot(x='time', y='total_bill', data=df, palette="Set3")
plt.xlabel('Período (Time)')
plt.ylabel('Média da Gorjeta')
plt.title('Média da Gorjeta por Período (Almoço ou Jantar)')
plt.show()
```

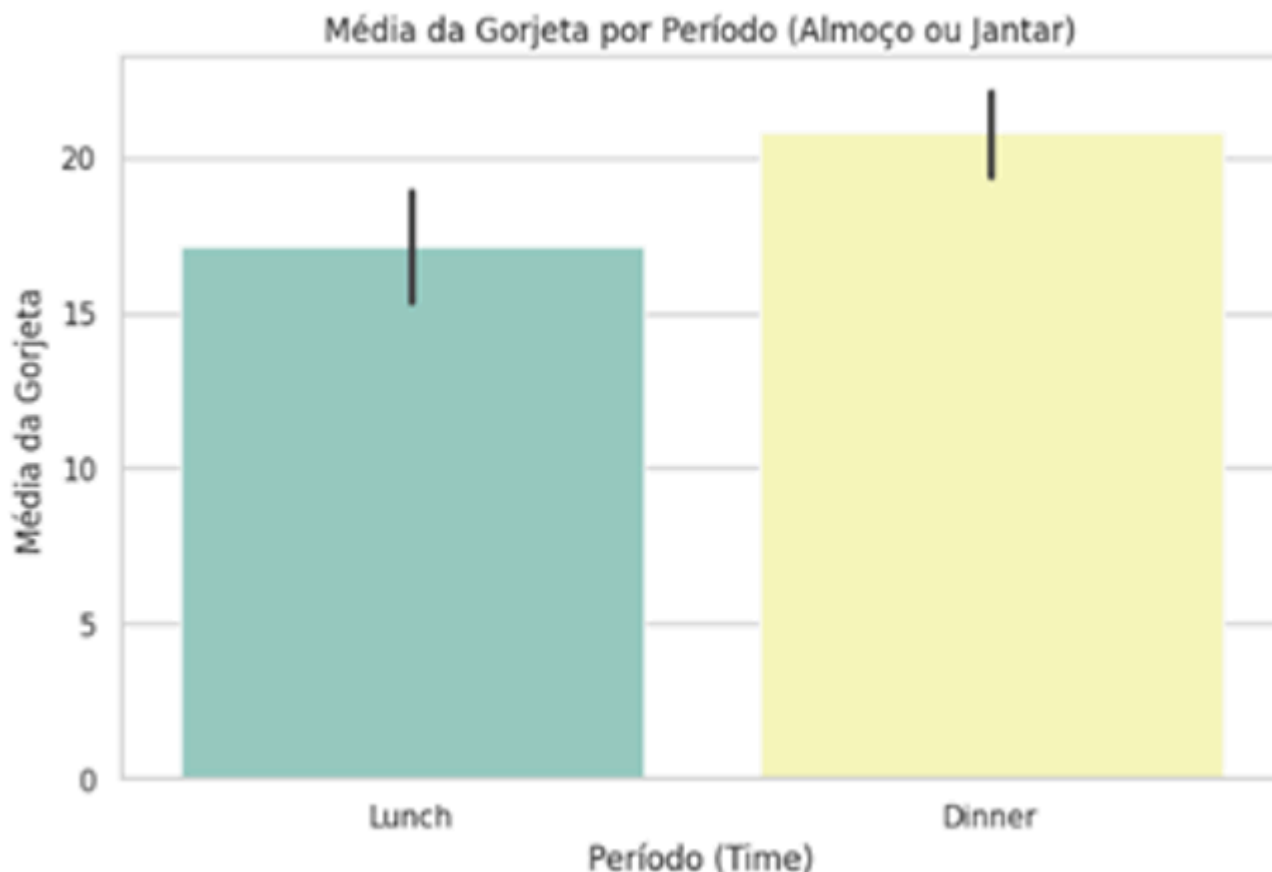


Figura 6 | Média de gorjeta por período (almoço ou janta). Fonte: elaborada pelo autor.

A média de gorjeta no jantar também é superior à do almoço.

Respondemos graficamente às perguntas feitas. Como mencionado anteriormente, a prática é sempre muito importante. Por isso, pense em soluções alternativas, mude parâmetros e se divirta no mundo do Python gráfico.

Saiba mais

1. A comunicação no mundo dos dados é tão importante quanto saber lidar com eles, por isso sugiro a leitura do capítulo 10 do livro [Comunicação inteligente e storytelling: para alavancar negócios e carreiras](#), cujo link de acesso está disponível a seguir.

ARRUDA, R. **Comunicação inteligente e storytelling**: para alavancar negócios e carreiras. Rio de Janeiro: Alta Books, 2019. E-book.

2. Outra dica para estudo e aprofundamento sobre esse tema é o livro [Use a cabeça! Python](#).

BARRY, P. **Use a cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book.

3. Para a aplicação do Python em data science, indico a leitura do capítulo 3 do livro [Data science do zero](#). Nesse texto, é possível entender como a visualização dos dados é importante para contar a história dos dados.

GRUS, J. **Data science do zero: primeiras regras com o Python**. Rio de Janeiro: Alta Books, 2021. E-book.

Referências

API reference. **Seaborn**, 20 out. 2016. Disponível em: <https://seaborn.pydata.org/api.html>. Acesso em: 5 nov. 2023.

ARRUDA, R. **Comunicação inteligente e storytelling: para alavancar negócios e carreiras**. Rio de Janeiro: Alta Books, 2019. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788550812977/>. Acesso em: 5 nov. 2023.

BARRY, P. **Use a cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

GRUS, J. **Data science do zero: primeiras regras com o Python**. Rio de Janeiro: Alta Books, 2021. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788550816463>. Acesso em: 12 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29. ed. São Paulo: Érica, 2019.

MWASKOM/SEABORN-DATA. **GitHub**, [s. d.]. Disponível em: <https://github.com/mwaskom/seaborn-data>. Acesso em: 5 nov. 2023.

PANDAS.DATAFRAME.PLOT. **pandas**, [s. d.]. Disponível em: <https://pandas.pydata.org/pandasdocs/stable/reference/api/pandas.DataFrame.plot.html>. Acesso em: 5 nov. 2023.

Aula 5

Introdução à Análise de Dados com Python

Videoaula de Encerramento

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Chegada

Olá, estudante! Para desenvolver a competência associada a esta unidade de aprendizagem, que é “Compreender os principais recursos de banco de dados e bibliotecas na linguagem Python”, devemos, antes de tudo, conhecer os conceitos relacionados à linguagem SQL, saber fazer conexões com bancos de dados e manipular tais dados dentro do banco de dados.

Ao longo desta etapa de estudos, foi possível aliar os assuntos estudados à prática de diversas formas. Você também aprendeu a utilizar a biblioteca pandas, que é uma das mais usadas na área de manipulação de dados em Python (Manzano; Oliveira, 2019). Saber utilizar bibliotecas como o pandas é de extrema importância para desfrutar do potencial máximo da linguagem Python.

Ao tratar de manipulação de dados, devemos ter em mente que muitas vezes os dados não se apresentam da forma mais padronizada possível, o que dificulta a extração de informações úteis. Por essa razão, a manipulação é uma parte do processo com a qual se deve ter muito cuidado (Perkovic, 2016).

Durante esta trajetória de aprendizagem, você não apenas internalizou os princípios e técnicas apresentados, mas também os colocou em ação em cenários do mundo real. Elaborar uma “história” dos dados com o auxílio gráfico do Python não apenas amplia seu domínio sobre essa ferramenta, mas também aprimora sua capacidade de desmembrar problemas intrincados em passos lógicos e de conceber soluções algorítmicas para resolvê-los, a fim de exibir os dados de modo simplificado.

Esse conjunto de habilidades o capacitará para se tornar um solucionador experiente de desafios tecnológicos e computacionais, preparando-o para enfrentar problemas de maneira eficaz. Agora você já possui o conhecimento necessário sobre o uso da visualização gráfica para aprimorar suas competências como programador.

É Hora de Praticar!



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Para contextualizar sua aprendizagem, imagine a seguinte situação: você está desenvolvendo um programa de gerenciamento de informações sobre funcionários na tabela de um banco de dados SQLite.

Questões norteadoras:

1. Como você pode aplicar seus conhecimentos em programação em Python para gerenciar essas informações?
2. Como é possível criar o banco de dados e atualizá-lo utilizando os conceitos de Python?

Para encerrar e consolidar seu aprendizado, reflita sobre as seguintes perguntas:

1. Aprender sobre SQL e conexão com o banco de dados me ajuda a ser um programador mais completo?
2. Qual é a importância das bibliotecas “prontas” da Python, como o pandas, na manipulação de dados?
3. Como a visualização gráfica dos dados ajuda a contar a “história” dos dados e a tomar decisões?

Essas considerações ajudarão você a incorporar de maneira mais profunda o conhecimento adquirido e a compreender o alcance de suas aplicações.

Desejo a você muito sucesso em sua jornada de aprendizagem!

Conectar ao banco de dados SQLite

Primeiro, precisamos criar um banco de dados SQLite e uma tabela para armazenar as informações dos funcionários. Vamos estabelecer uma conexão com o banco de dados.

Confira, a seguir, o código Python a ser usado:

```
import sqlite3
# Passo 1: Conectar ao banco de dados SQLite (ou criá-lo, se não existir)
conn = sqlite3.connect("funcionarios.db")
```

Criar a tabela de funcionários

Vamos criar uma tabela chamada "funcionarios" para armazenar as informações dos funcionários.

```
# Passo 2: Criar a tabela de funcionários
cursor = conn.cursor()
cursor.execute("""
    CREATE TABLE IF NOT EXISTS funcionarios (
        id INTEGER PRIMARY KEY,
        nome TEXT,
        cargo TEXT,
        salario REAL
    )
""")
```

Inserir um novo funcionário

Agora, vamos inserir um novo funcionário na tabela, simulando a operação de "Create".

```
# Passo 3: Inserir um novo funcionário na tabela
novo_funcionario = (1, "João", "Analista", 5000.00)
cursor.execute("INSERT INTO funcionarios VALUES (?, ?, ?, ?)", novo_funcionario)
conn.commit()
```

Consultar funcionários

Podemos consultar os funcionários existentes na tabela, simulando a operação de "Read".

```
# Passo 4: Consultar e exibir funcionários
cursor.execute("SELECT * FROM funcionarios")
funcionarios = cursor.fetchall()
print("Funcionários Cadastrados:")
for funcionario in funcionarios:
    print(funcionario)
```

Atualizar informações de um funcionário

Agora, vamos atualizar as informações de um funcionário específico, simulando a operação de "Update".

```
# Passo 5: Atualizar informações de um funcionário
atualizacao = ("João Silva", 5500.00, 1)
cursor.execute("UPDATE funcionarios SET nome = ?, salario = ? WHERE id = ?", atualizacao)
conn.commit()
```

Deletar um funcionário

Por fim, vamos deletar um funcionário da tabela, simulando a operação de "Delete".

```
# Passo 6: Deletar um funcionário da tabela
id_funcionario_para_deletar = 1
cursor.execute("DELETE FROM funcionarios WHERE id = ?", (id_funcionario_para_deletar,))
conn.commit()
```

Esse é um exemplo desenvolvido em um ambiente real. Você deve implementar mecanismos de tratamento de erros e segurança adequados para lidar com operações em um banco de dados. As operações CRUD são fundamentais na gestão de dados em sistemas que utilizam bancos de dados relacionais como o SQLite.

O material visual a seguir esquematiza os principais tópicos abordados nesta unidade de aprendizagem, que apresentou uma introdução à análise com Python. Este infográfico exibe uma percepção clara e sucinta de cada parte desta etapa de estudos, enfatizando os conceitos e fundamentos necessários para uma boa compreensão dos saberes desenvolvidos.

TIMELINE

INTRODUÇÃO À ANÁLISE DE DADOS COM PYTHON



Figura 1 | Infográfico: explorando recursos do Python. Fonte: elaborada pelo autor.

ARRUDA, R. **Comunicação inteligente e storytelling**: para alavancar negócios e carreiras. Rio de Janeiro: Alta Books, 2019. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/#/books/9788550812977/>. Acesso em: 5 nov. 2023.

GRUS, J. **Data science do zero**: primeiras regras com o Python. Rio de Janeiro: Alta Books, 2021. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/#/books/9788550816463/>. Acesso em: 12 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos**: lógica para desenvolvimento de programação de computadores. 29. ed. São Paulo: Érica, 2019.

PERKOVIC, L. **Introdução à computação usando Python**: um foco no desenvolvimento de aplicações. Rio de Janeiro: LTC, 2016.